

Data Pre-processing

Data pre-processing

- Data preprocessing in Machine Learning refers to the technique of preparing (cleaning and organizing) the raw data to make it suitable for a building and training Machine Learning models.
- It is a data mining technique that transforms raw data into an understandable and readable format. Data pre-processing is the first and crucial step while creating a machine learning model. When creating a machine learning project, it is not always a case that we come across the clean and formatted data. Hence in order to increase the accuracy and perform of the model data pre-processing is important step.

Importance of data pre-processing in ML

- ✓ A real-world data generally contains noises, missing values, and maybe in an unusable format which cannot be directly used for machine learning models.
- ✓ Data preprocessing is required tasks for cleaning the data and making it suitable for a machine learning model which also increases the accuracy and efficiency of a machine learning model.
- ✓ Data preprocessing is an integral step in Machine Learning as the quality of data and the useful information that can be derived from it directly affects the ability of our model to learn; therefore, it is extremely important that we preprocess our data before feeding it into our model.

Data cleaning

- ▶ Data cleaning is the process of fixing or removing incorrect, corrupted, incorrectly formatted, duplicate, or incomplete data within a dataset.
- ▶ When combining multiple data sources, there are many opportunities for data to be duplicated or mislabeled.
- ▶ Data cleaning means fixing bad data in your data set.
 - Bad data could be:
 - Empty cells
 - Data in wrong format
 - Wrong data
 - Duplicate

What is the difference between data cleaning and data transformation?

- ▶ Data cleaning is the process that removes data that does not belong in your dataset.
- ▶ Data transformation is the process of converting data from one format or structure into another.

Importance of data cleaning

- Removal of errors when multiple sources of data are used.
- Fewer errors make for happier clients and less-frustrated employees.
- Ability to map the different functions and what your data is intended to do.
- Monitoring errors and better reporting to see where errors are coming from, making it easier to fix incorrect or corrupt data for future applications.
- Using tools for data cleaning will make for more efficient business practices and quicker decision-making.

Data Anomaly

- Anomalies are data points that stand out amongst other data points in the dataset and do not confirm the normal behavior in the data.
- These data points or observations deviate from the dataset's normal behavioral patterns.
- In data analysis, anomaly detection is generally understood to be the identification of rare items, events or observations which deviate significantly from the majority of the data and do not confirm to a well-defined notion of normal behavior.
- Simple statistical techniques such as mean, median, quantiles can be used to detect univariate anomalies feature values in the dataset. Various data visualization and exploratory data analysis techniques can be also be used to detect anomalies.

Detect missing values with pandas DataFrame functions: .info() and .isna()

Missing values are those values that are not present in the dataset. Some values of the attributes are empty due to various reasons. The empty values/null values in the dataset are termed as missing values.

The real-world data often has a lot of missing values. The cause of missing values can be data corruption or failure to record data. The handling of missing data is very important during the pre-processing of the dataset as many machine learning algorithms do not support missing values.

1. **.info()** method in pandas DataFrame prints information about the DataFrame. The information contains the number of columns, column labels, column data types, memory usage, range index, and the number of cells in each column (non-null values).

Note: the info () method prints the info.

- .info()– the pandas functionality.

Import pandas as pd

df=pd.read_csv("nba.csv")

Print(df)

Df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 458 entries, 0 to 457
Data columns (total 9 columns):
Name                457 non-null object
Team                457 non-null object
Number              457 non-null float64
Position            457 non-null object
Age                 457 non-null float64
Height              457 non-null object
Weight              457 non-null float64
College             373 non-null object
Salary              446 non-null float64
dtypes: float64(4), object(5)
memory usage: 32.3+ KB
```

As we can see it allows us to see the summary of the dataset crisp and short.it helps us when we have 1000s of attributes and rows in the dataset.

2. **.isna()** function is used to check the presence of Null values in the dataset.

	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	Jae Crowder	Boston Celtics	99.0	SF	25.0	6-6	235.0	Marquette	6796117.0
2	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston University	NaN
3	R.J. Hunter	Boston Celtics	28.0	SG	22.0	6-5	185.0	Georgia State	1148640.0
4	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
5	Amir Johnson	Boston Celtics	90.0	PF	29.0	6-9	240.0	NaN	2000000.0
6	Jordan Mickey	Boston Celtics	55.0	PF	21.0	6-8	235.0	LSU	1170960.0
7	Kelly Olynyk	Boston Celtics	41.0	C	25.0	7-0	238.0	Gonzaga	2165160.0
8	Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
9	Marcus Smart	Boston Celtics	36.0	PG	22.0	6-4	220.0	Oklahoma State	3431040.0
10	Jared Sullinger	Boston Celtics	7.0	C	24.0	6-9	260.0	Ohio State	2569260.0
11	Isaiah Thomas	Boston Celtics	4.0	PG	27.0	5-9	185.0	Washington	6912869.0
12	Evan Turner	Boston Celtics	11.0	SG	27.0	6-7	220.0	Ohio State	3425510.0
13	James Young	Boston Celtics	13.0	SG	20.0	6-6	215.0	Kentucky	1749840.0
14	Tyler Zeller	Boston Celtics	44.0	C	26.0	7-0	253.0	North Carolina	2616975.0
15	Bojan Bogdanovic	Brooklyn Nets	44.0	SG	27.0	6-8	216.0	NaN	3425510.0
16	Markel Brown	Brooklyn Nets	22.0	SG	24.0	6-3	190.0	Oklahoma State	845059.0

- The easiest way to check for missing values in a Pandas DataFrame is via the isna() function.
- The isna() function returns a Boolean (True or False) value if the Pandas column value is missing, so if you run **df.isna()** you'll get back a DataFrame showing you a load of Boolean values.

Import pandas as pd

df=pd.read_csv("nba.csv")

Print(df)

Types of missing values

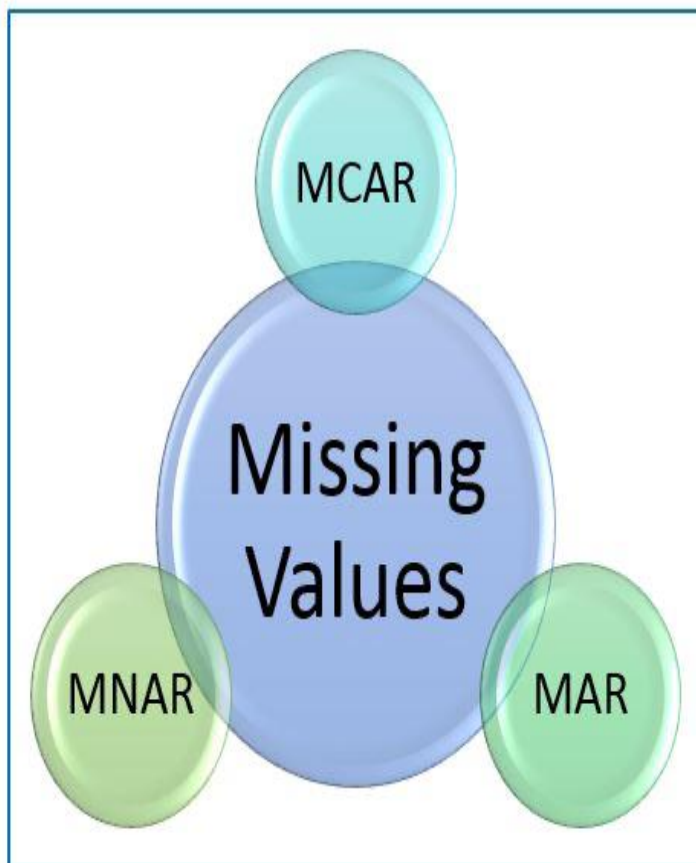


Figure 1 - Different Types of Missing Values in Datasets

1. Missing Completely At Random (MCAR)

- In MCAR, the probability of data being missing is the same for all the observations.
- In this case, there is no relationship between the missing data and any other values observed or unobserved (the data which is not recorded) within the given dataset.
- That is, missing values are completely independent of other data. There is no pattern.
- In the case of MCAR, **the data could be missing due to human error, some system/equipment failure, loss of sample, or some unsatisfactory technicalities while recording the values.**
- For Example, suppose in a library there are some overdue books. Some values of overdue books in the computer system are missing. The reason might be a human error like the librarian forgot to type in the values. So, the missing values of overdue books are not related to any other variable/data in the system.

- It should not be assumed as it's a rare case. The advantage of such data is that the statistical analysis remains unbiased.

2. Missing At Random (MAR)

- Missing at random (MAR) means that the reason for missing values can be explained by variables on which you have complete information as there is some relationship between the missing data and other values/data.
- In this case, the data is not missing for all the observations. It is missing only within sub-samples of the data and there is some pattern in the missing values.
- For example, if you check the survey data, you may find that all the people have answered their 'Gender' but 'Age' values are **mostly** missing for people who have answered their 'Gender' as 'female'. (The reason being most of the females do not want to reveal their age.)
- So, the probability of data being missing depends only on the observed data.
- In this case, the variables 'Gender' and 'Age' are related and the reason for missing values of the 'Age' variable can be explained by the 'Gender' variable but you cannot predict the missing

3. Missing Not At Random (MNAR)

- Missing values depend on the unobserved data.
- If there is some structure/pattern in missing data and other observed data **cannot explain** it, then it is **Missing Not At Random (MNAR)**.
- If the missing data does not fall under the MCAR or MAR then it can be categorized as MNAR.
- It can happen due to the reluctance of people in providing the required information. A specific group of people may not answer some questions in a survey.
- For example, suppose the name and the number of overdue books is asked in the poll for a library. So, most of the people having no overdue books are likely to answer the poll. People having more overdue books are less likely to answer the poll.
- So, in this case, the missing value of the number of overdue books depends on the people who have more books overdue.
- Another example, people having less income may refuse to share that information in a survey.
- In the case of MNAR as well the statistical analysis might result in bias.

Approaches to deal with missing values

- Keep the missing value as is.
- Remove data objects with missing values.
- Remove the attributes with missing values.
- Estimate and impute missing values.

Analyse each column with missing values carefully to understand the reasons behind the missing values as it is crucial to find out the strategy for handling the missing values.

There are 2 primary ways of handling missing values:

1. Deleting the Missing values
2. Imputing the Missing Values

Program to find missing values, deleting missing value rows, dropping columns and imputing missing values.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
data=pd.read_csv("C:/Users/Shilpa/Desktop/dataset/titanic/train.csv")
print(data.dtypes)
print(data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId     891 non-null    int64
1   Survived        891 non-null    int64
2   Pclass          891 non-null    int64
3   Name            891 non-null    object
4   Sex             891 non-null    object
5   Age             714 non-null    float64    # contains missing values
6   SibSp           891 non-null    int64
7   Parch           891 non-null    int64
8   Ticket          891 non-null    object
9   Fare            891 non-null    float64
10  Cabin           204 non-null    object
11  Embarked        889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

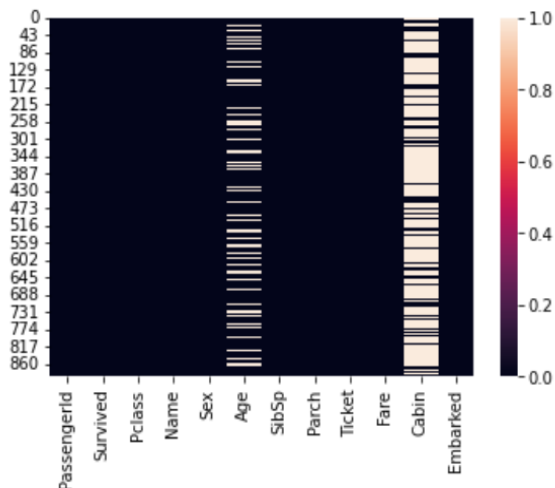
```
Data.isnull.sum()
print(data.isna().sum())
print(data.notnull().sum())
print(data['Age'].isna().sum())
```

```
print(data.isnull().sum())
```

```
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age             177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin           687
Embarked         2
dtype: int64
```

```
In [3]: sns.heatmap(data.isna())
```

```
Out[3]: <AxesSubplot:>
```



```
print(data.dropna(inplace=True))
print(data.isnull().sum())
```

#dropping rows of the missing values

```
datacol=data.drop(['Cabin'],axis=1)
print(datacol)
sns.heatmap(data.isna())
```

#dropping columns of the missing values

Imputing the Missing Value: imputing missing values means

Program to demonstrate imputing missing values in dataset.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
data=pd.read_csv("C:/Users/Shilpa/Desktop/dataset/titanic/train.csv")
```

Imputing age value with mean

```
print(data.isnull().sum())
avgage=data['Age'].mean()
print(avgage)
```

```
data['Age'].fillna(avgage,inplace=True)
print(data["Age"].isna().sum())
```

Imputing age value with mode

```
data['Embarked']=data['Embarked'].fillna(data['Embarked'].mode()[0])
data['Age']=data['Age'].fillna(data['Age'].median())
```

Outliers

An outlier is an observation that appears far way and diverges from an overall pattern in a sample.

- In statistics, an outlier is a data point that differs significantly from other observations.
- An outlier may be due to variability in the measurement or it may indicate experimental error; the latter are sometimes excluded from the data set. An outlier can cause serious problems in statistical analyses.
- The outlier increases the error variance and reduces the power of statistical tests.

What is an outlier example?

A value that "lies outside" (is much smaller or larger than) most of the other values in a set of data.

For example, in the scores 25,29,3,32,85,33,27,28 both 3 and 85 are "outliers"

Outliers Identification

There are different ways and methods of identifying outliers and most popular techniques among them are as follows:

- ▶ Visualization
- ▶ Skewness
- ▶ Interquartile Range
- ▶ Standard Deviation

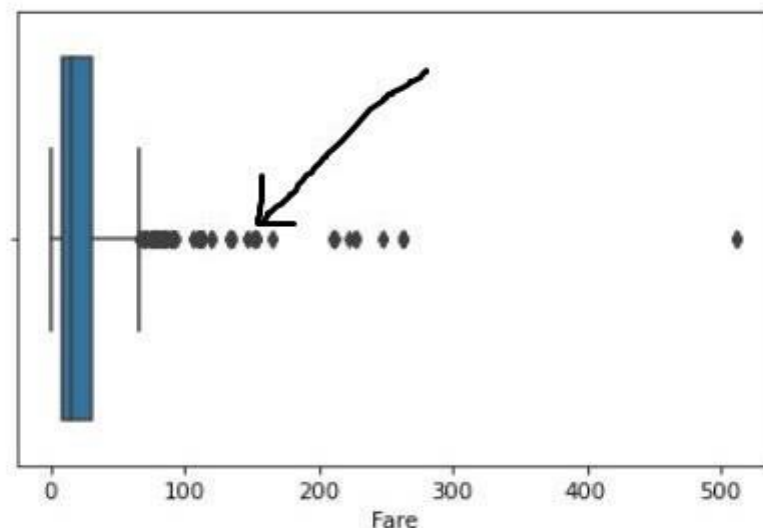
Visualization

Outliers can be detected using different visualization methods, we are going to use :

- ▶ Boxplot
- ▶ Histogram

1. Boxplot

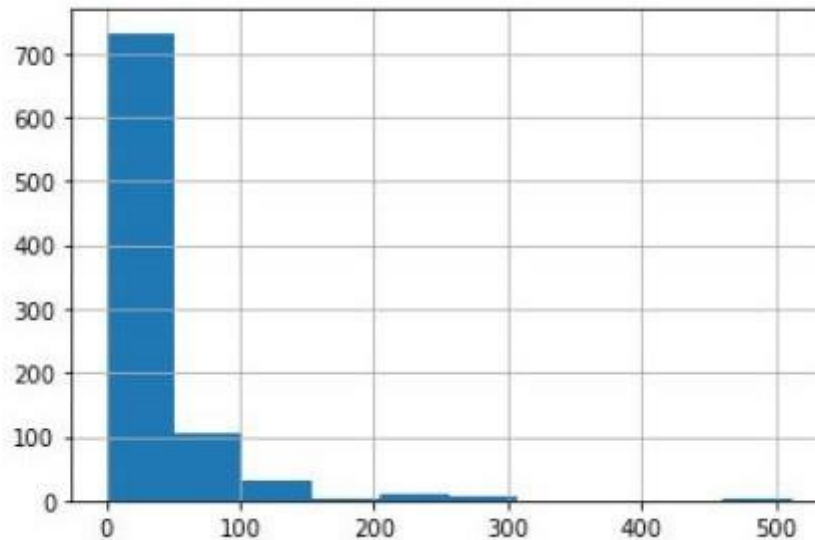
boxplot is a visualization tool for identifying outliers, it displays the distribution of statistical observations, its body is classified into four parts; the lowest and the highest (minimum and maximum), the 25 percentile (first quartile(Q1)), the median (50th percentile), the 75th percentile (third quartile(Q3)). outliers appears above or below the minimum and maximum of the boxplot.



As we can see from the above figure that the points lying above the maximum are outliers.

2. Histogram

To visualize the distribution of a numerical variable, a histogram shows the direction in which these variables are distributed, outliers will appear outside the overall distribution of the data. If the histogram is right-skewed or left-skewed, it indicates the presence of extreme values or outliers.



3. Skewness

- The skewness value should be within the range of -1 to 1 for a normal distribution, any major changes from this value may indicate the presence of outliers.

From the titanic dataset

```
print('skewness value of Age: ',df['Age'].skew())  
print('skewness value of Fare: ',df['Fare'].skew())
```

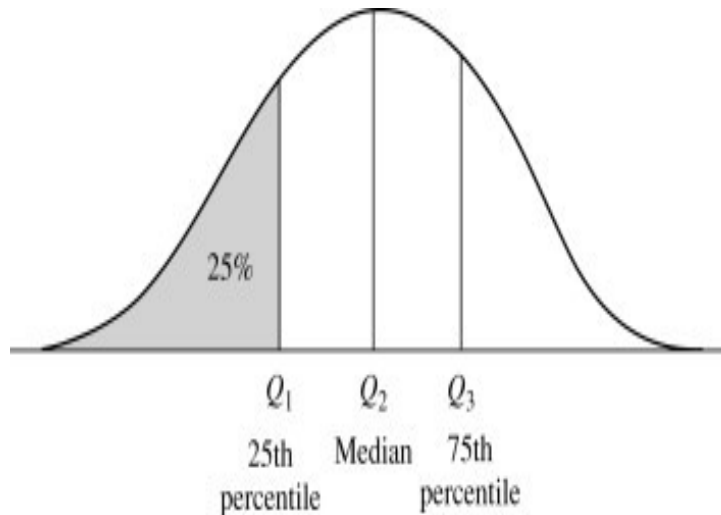
output

```
skewness value of Age: 0.6577529069911331  
skewness value of Fare: 4.787316519674893
```

- the skewness value should be within the range of -1 to 1 for a normal distribution, any major changes from this value indicates the presence of extreme value or outlier.
- from the code above, the 'Fare' skewness value of 4.78 shows the variable has been rightly skewed, indicating the presence of outliers.

4. Interquartile Range (IQR)

The interquartile range is a measure of statistical dispersion and is calculated as the difference between 75th and 25th percentiles. The Quartiles divide the data set into four equal parts. The values that separate parts are called the first, second, and third quartiles.



The outliers must be removed and there are various ways of treating outlier.

Dealing with outliers

- Do nothing
- Replace with the upper cap or lower cap
- Perform a log transformation
- Remove data objects with outliers

All the above methods are demonstrated through a program given below

Program to demonstrate detection of outlier and removing it with various methods

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

#Loading dataset, looking datatypes and statistical analysis

```
titanic=pd.read_csv("C:/Users/Shilpa/Desktop/dataset/titanic/train.csv")
print(titanic.head())
print(titanic.info())
print(titanic.describe())
```

#Percentage of missing values

```
missing_values=titanic.isnull().sum()
print(missing_values[missing_values>0]/len(titanic)*100)
```

#filling missing values

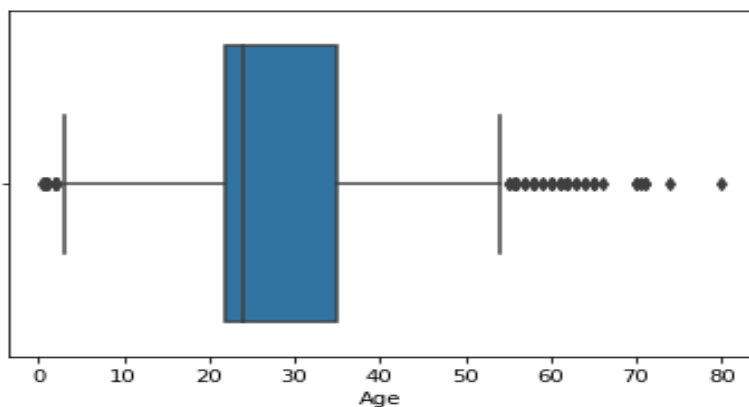
```
titanic['Age']=titanic['Age'].fillna(titanic['Age'].mode()[0])
titanic['Embarked']=titanic['Embarked'].fillna(titanic['Embarked'].mode()[0])
```

#Removing unnecessary columns

```
titanic.drop(['PassengerId','Name','Ticket','Cabin'],axis=1,inplace=True)
print(titanic.head())
```

#To find outliers in age column

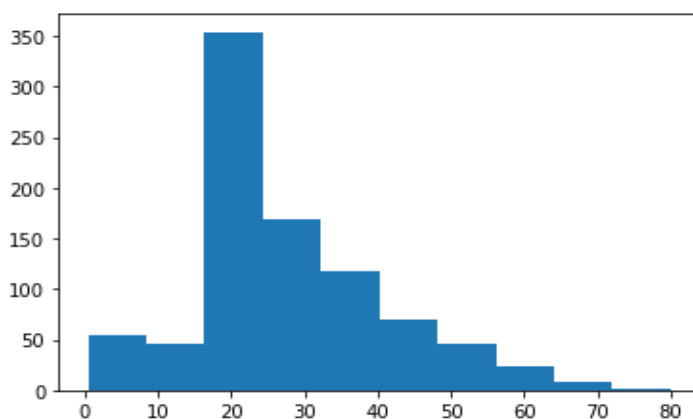
```
sns.boxplot(titanic['Age'])
```



```
Print(np.where(titanic["Age"]>55))    #give the places of the outlier
```

#finding outlier through histogram graph

```
plt.hist(titanic["Age"])
```

**#finding outlier through skewness**

```
print('skewness value of Age: ',titanic['Age'].skew())
```

#finding through inter quartile range

```
Q1=titanic['Fare'].quantile(0.25)
```

```
Q3=titanic['Fare'].quantile(0.75)
```

```
IQR=Q3-Q1
```

```
print("----",IQR)
```

```
whisker_width = 1.5
```

```
Fare_outliers = titanic[(titanic['Fare'] < Q1 - whisker_width*IQR) | (titanic['Fare'] > Q3 +  
whisker_width*IQR)]
```

```
print(Fare_outliers.head())
```

Program to remove outlier from dataset

```
import pandas as pd
```

```
import numpy as np
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
titanic=pd.read_csv("C:/Users/Shilpa/Desktop/dataset/titanic/train.csv")
```

```
Q1 = titanic['Age'].quantile(0.25)
```

```
Q3 = titanic['Age'].quantile(0.75)
```

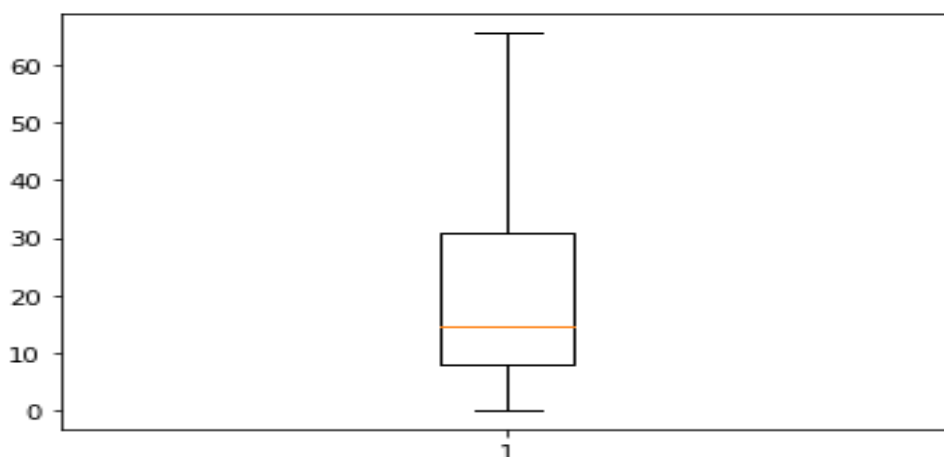
```
IQR = Q3 - Q1
```

```
lower_whisker = Q1 - (1.5*IQR)
```

```
upper_whisker = Q3 + (1.5*IQR)
```

```
titanic['Age']=np.where(titanic['Age']>upper_whisker,upper_whisker,np.where(titanic['Age']<lower_  
_whisker,lower_whisker,titanic['Age']))
```

```
plt.boxplot(titanic["Age"])
```



Now u can see in the above graph there is no outliers.

A program to detect and remove outlier for loan dataset

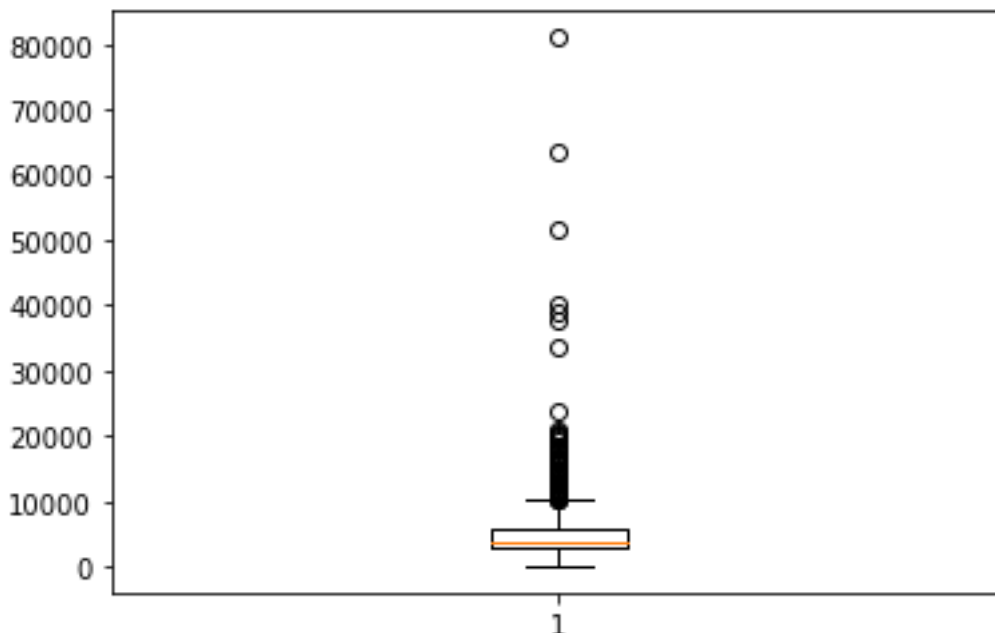
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

df=pd.read_csv("C:/Users/Shilpa/Desktop/dataset/loan-train.csv")
print(df.shape)
print(df.info())
print(df.describe())

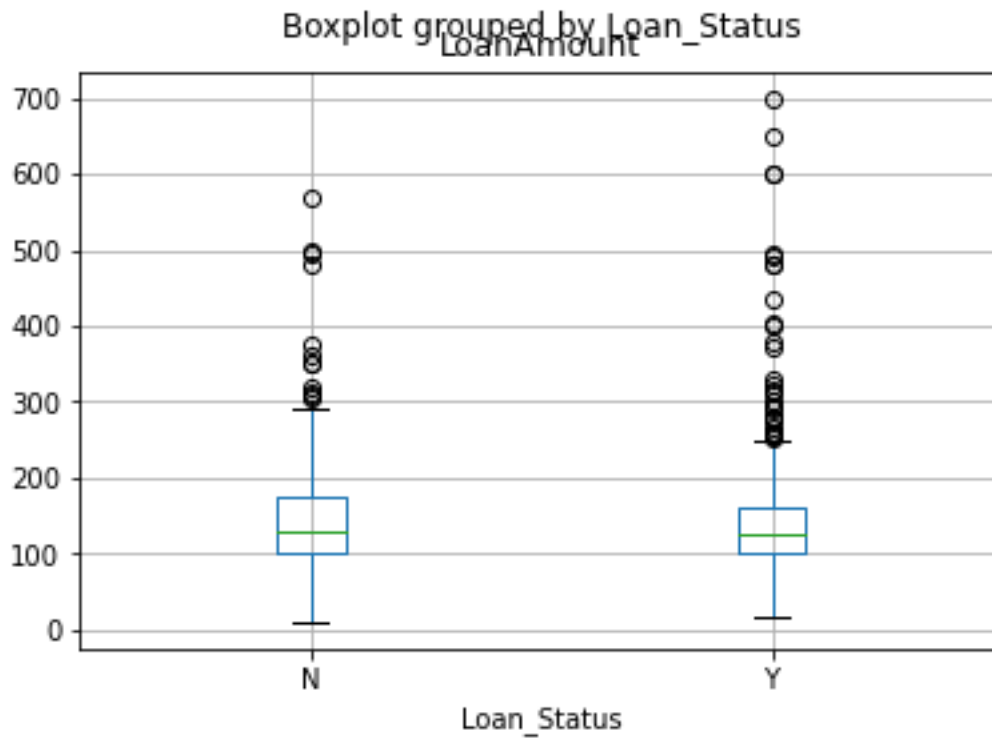
# identify outlier using IQR
Q1 = df.quantile(0.25)
Q3 = df.quantile(0.75)
IQR = Q3 - Q1
print(IQR)

print(df['ApplicantIncome'].skew())
df['ApplicantIncome'].describe()

#outlier using boxplot
plt.boxplot(df['ApplicantIncome'])
```

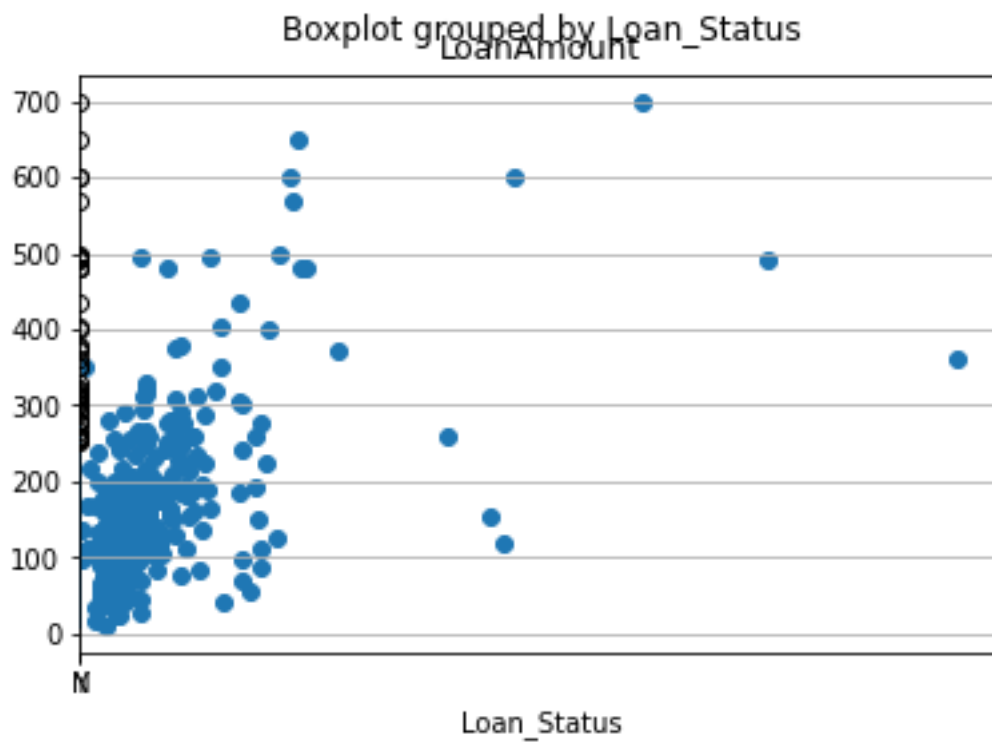


```
df.boxplot(column='LoanAmount', by='Loan_Status')
```



```
#bivariate outlier
```

```
plt.scatter(df['ApplicantIncome'],df['LoanAmount'])
```



```
df["ApplicantIncome"] = np.where(df["ApplicantIncome"] < 2960.0, 2960.0, df['ApplicantIncome'])

df["ApplicantIncome"] = np.where(df["ApplicantIncome"] > 12681.0,
12681.0, df['ApplicantIncome'])
print(df['ApplicantIncome'].skew()) # skew value reduced from 6 to 1.4
```

#trimming method to remove outlier

```
Q1 = df['ApplicantIncome'].quantile(0.10)
Q3 = df['ApplicantIncome'].quantile(0.90)
IQR = Q3 - Q1
whisker_width = 1.5
lower_whisker = Q1 - (whisker_width*IQR)
upper_whisker = Q3 + (whisker_width*IQR)
index = df['ApplicantIncome'][(df['ApplicantIncome'] > upper_whisker) | (df['ApplicantIncome'] < lower_
_whisker)].index
df.drop(index, inplace=True)
```

#again removing the outlier where application income is more than 10000

```
index = df[(df['ApplicantIncome'] >= 10000) | (df['ApplicantIncome'] <= 1000)].index
df.drop(index, inplace=True)
df['ApplicantIncome'].describe()

plt.boxplot(df['ApplicantIncome'])
```

Dropping outliers

Trimming

in this method, we removed and completely drop all the outliers,
the line of code below creates an index for all data points and drop the index values.

```
Q1 = df['Fare'].quantile(0.10)
Q3 = df['Fare'].quantile(0.90)
IQR = Q3 - Q1
whisker_width = 1.5
lower_whisker = Q1 - (whisker_width*IQR)
upper_whisker = Q3 + (whisker_width*IQR)
index = df['Fare'][(df['Fare'] > upper_whisker) | (df['Fare'] < lower_whisker)].index
df.drop(index, inplace=True)
```

Data integration

What is data integration in ML?

Data Integration refers to the process of unifying data from multiple data sources.

- This task involves integrating data from multiple sources such as databases (relational and non-relational), data cubes, files, etc. The data sources can be homogeneous or heterogeneous. The data obtained from the sources can be structured, unstructured or semi-structured in format.
- It is a strategy that integrates data from several sources to make it available to users in a **single uniform view that shows their status.**

1.Program to create two dataset and merge on student_id attribute

```
import pandas as pd
import numpy as np

stud=pd.read_csv("C:/Users/Shilpa/Desktop/dataset/students.csv")
print(stud.head(10))

marks=pd.read_csv("C:/Users/Shilpa/Desktop/dataset/marks.csv")
print(marks.head(10))

df=pd.merge(stud,marks)
print(df.head(10))
```

2.program to add new column and inserting new attribute at particular index in dataset

```
import pandas as pd
import numpy as np

stud=pd.read_csv("C:/Users/Shilpa/Desktop/dataset/students.csv")
#print(stud.head(10))

tutors=["shilpa","veena","vani","roopa","shilpa","rekha"]
college=["sit","spt","sit","spt","sit","sit"]

add=stud.assign(totursassigned=tutors,collagename=college)
print(add)

total=[450,600,467,567,560,456]
new=add.assign(totalmarks=total)
```

```
print(new)
```

```
new=new.assign(percantage=lambda x: new.totalmarks *100 /600)  
print(new)
```

```
new=new.assign(a=None,b=0)  
print(new)
```

```
new1=new.insert(1,"tutors_indexing",tutors)  
print(new)
```

data Integration challenges

- *Keeping up with data volumes*

The biggest data integration challenge is the exponential growth of data from many different sources. This causes problems with the available capacity for data retention and, more important, efforts to make actionable information out of all the data being created and collected.

- *Different streaming data sources*

Many enterprises struggle to integrate data from disparate sources.

- *Manual data integration*

In many companies, roughly 80% of a data scientist's time is spent finding and prepping data, leaving only the equivalent of one day a week for actual data science work, Enterprises may want to evaluate tools to automate data ingestion and integration. They can be used to create and document integration jobs, make consistent changes to all the ingested data, and position it in a standard way.

Data Reduction:

- Data reduction is a process that reduces the volume of original data and represents it in a much smaller volume.
- Data reduction techniques are used to obtain a reduced representation of the dataset that is much smaller in volume by maintaining the integrity of the original data.
- By reducing the data, the efficiency of the data mining process is improved, which produces the same analytical results.
- The method of data reduction may achieve a condensed description of the original data which is much smaller in quantity but keeps the quality of the original data.
- Data reduction can increase storage efficiency and performance and reduce storage costs.

Distinction between data reduction and data redundancy

Data redundancy means duplicate data and it is also stated that the same parts of data exist in multiple locations into the database. This condition is known as Data Redundancy.

Whereas Data Reduction technique is used to reduce redundancy for better storage efficiency and performance.

Problems with Data Redundancy:

Here, we will discuss the few problems with data redundancy as follows.

1. Wasted Storage Space.
2. More Difficult Database Update.
3. It will lead to Data Inconsistency.
4. Retrieval of data is slow and inefficient.

Example

Let us take an example of a cricket player table.

Step-1:

Consider cricket player table as follows.

Player Name	Player Age	Team Name	Team ID
Virat Kohli	32	India	1
Rohit Sharma	34	India	1
Ross Taylor	37	New Zealand	2
Shikhar Dhawan	35	India	1
Kane Williamson	30	New Zealand	2

Step-2 :

We can clearly see that the Team Name and Team Id are repeated at multiple places. we can make a separate table to store this information and reduce data redundancy.

Player Name	Player Age	Team Id
Virat Kohli	32	1
Rohit Sharma	34	1
Ross Taylor	37	2
Shikhar Dhawan	35	1
Kane Williamson	30	2

Step-3 :

This is known as Normalization used to reduce Data Redundancy.

Team Id	Team Name
1	India
2	New Zealand

Program to demonstrate Data Redundancy in Mall_Customers Dataset

```
import pandas as pd
import numpy as np
dfCustomers = pd.read_csv('C:/Users/Hp/OneDrive/Desktop/Datasets/Mall_Customers.csv')
dfCustomers.shape
```

```
(250, 5)
```

```
dfCustomers.info
```

```
<bound method DataFrame.info of
0      1  Male  19      15      39
1      1  Male  19      15      39
2      3  Female 20      16       6
3      4  Female 23      16      77
4      5  Female 31      17      40
..     ...   ...   ...     ...     ...
245    246  Male  30     297      69
246    247  Female 56     311      14
247    248  Male  29     313      90
248    249  Female 19     316      32
249    250  Female 31     325      86
```

```
[250 rows x 5 columns]>
```

```
dfCustomers.duplicated(subset=None, keep='first').sum()
```

```
2
```

```
dfCustomers.drop_duplicates(subset=None, keep='first', inplace=True)
```

```
dfCustomers.shape
```

```
(248, 5)
```

Objectives

- To reduce difficulties in analysis of huge data.
- To **increase the storage efficiency**.
- **To reduce data storage and analysis costs.**
- To make information more compact. It is easier to use sophisticated and computationally expensive algorithms when the data amount is less.

Methods

1) Numerosity Reduction:

It is a data reduction technique that replaces the original data with a smaller representation of that data. It provides more manageable dataset.

There are two techniques for Numerosity Reduction:

Parametric and Non-Parametric methods.

Parametric Methods

Here, data is represented using some model. The model is used to estimate the data, so that only parameters of data are required to be stored, instead of actual data.

Regression and Log-Linear methods are used for creating such models.

Regression:

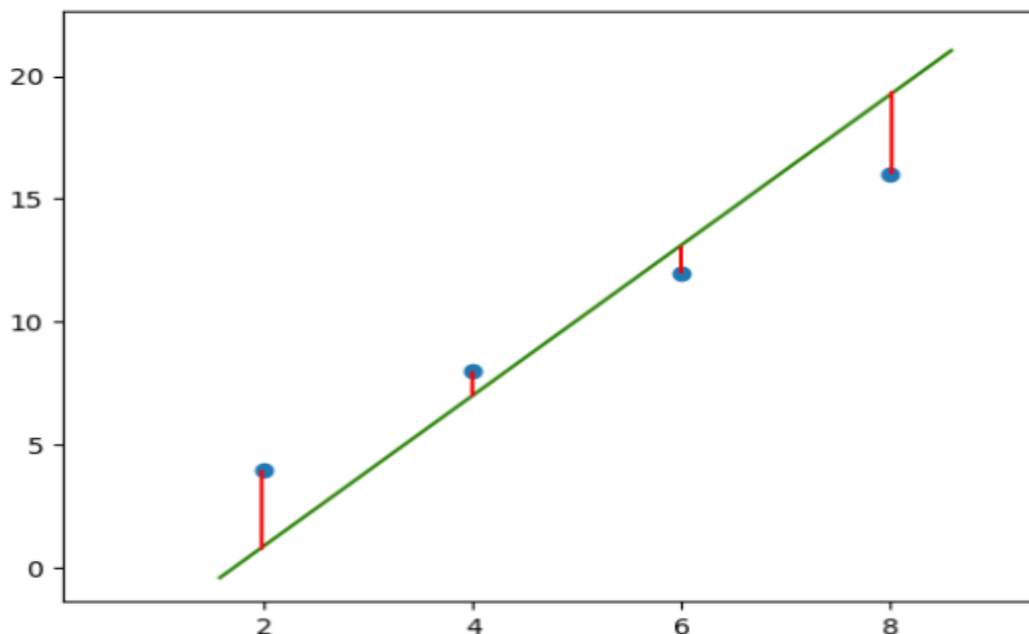
Regression can be a simple linear regression or multiple linear regression. When there is only single independent attribute, such regression model is called simple linear regression and if there are multiple independent attributes, then such regression models are called multiple linear regression.

In linear regression, the data are modelled to a fit straight line.

For example, a random variable y can be modelled as a linear function of another random variable x with the equation $y = ax + b$,

where a and b (regression coefficients) specifies the slope and y -intercept of the line, respectively.

In multiple linear regression, y will be modelled as a linear function of two or more predictor (independent) variables.



Non-Parametric Methods –

These methods are used for storing reduced representations of the data include *histograms*, *clustering*, *sampling* and *data cube aggregation*.

Histograms:

Histogram is the data representation in terms of frequency. It uses binning to approximate data distribution and is a popular form of data reduction.

Clustering:

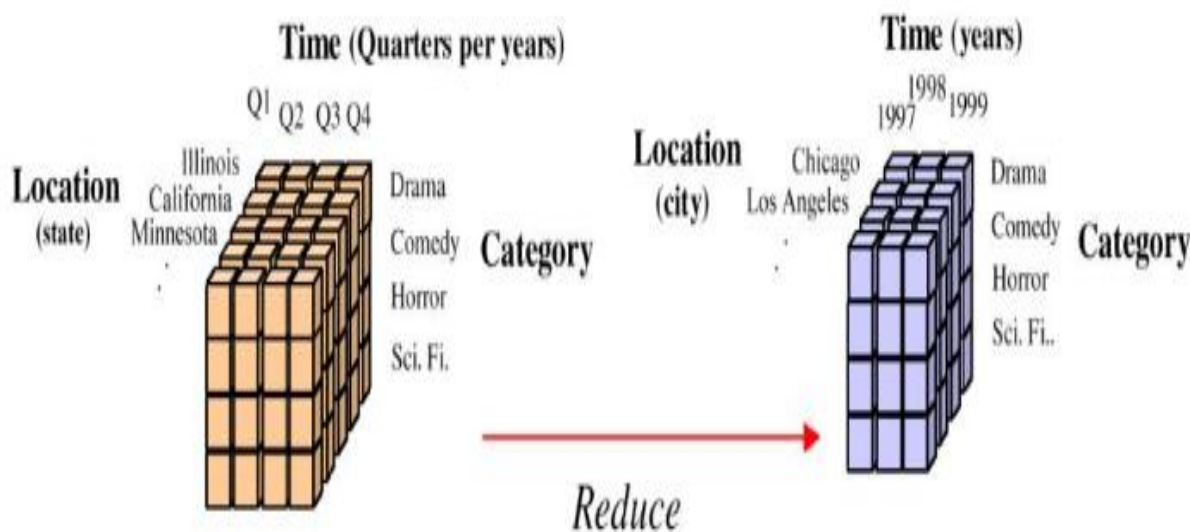
Clustering divides the data into groups/clusters. This technique partitions the whole data into different clusters. In data reduction, the cluster representation of the data is used to replace the actual data. It also helps to detect outliers in data.

Sampling:

Sampling can be used for data reduction because it allows a large data set to be represented by a much smaller random data sample (or subset).

Data Cube Aggregation:

Data cube aggregation involves moving the data from detailed level to a fewer number of dimensions. The resulting data set is smaller in volume, without loss of information necessary for the analysis task.

**2) Dimensionality Reduction:**

- This process is necessary for real-world applications as the data size is big.
- In this process, the reduction of random variables or attributes is done so that the dimensionality of the data set can be reduced. Combining and merging the attributes of the data without losing its original characteristics.
- This also helps in the reduction of storage space and computation time is reduced. This method uses data encoding mechanisms to further reduce its size.
- As with all compressing methods, you can go for a lossy or lossless option, depending on your specific needs and whether you want to retrieve the original information in its entirety or can afford the loss of certain parts.

Data Transformation

It includes Data Cleaning and Data reduction techniques to convert the raw data in to the appropriate form.

Need of Data Transformation

- It is an essential data pre-processing technique that must be performed on the data before data mining to provide patterns that are easier to understand.
- Data transformation changes the format, structure, or values of the data and converts them into clean, usable data.

Normalization

It rescales a dataset so that each value falls between 0 and 1. It uses the following formula to do so:

$$X_{\text{new}} = (X_i - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

where:

- x_i : The i^{th} value in the dataset
- x_{min} : The minimum value in the dataset
- x_{max} : The maximum value in the dataset

The following examples show how to standardize and normalize a dataset in practice.

suppose we have the following dataset:

Data Values
13
16
19
22
23
38
47
56
58
63
65
70
71

The minimum value in the dataset is 13 and the maximum value is 71.

To normalize the first value of **13**, we would apply the formula shared earlier:

- $X_{\text{new}} = (X_i - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}}) = (13 - 13) / (71 - 13) = 0$

To normalize the second value of **16**, we would use the same formula:

- $X_{\text{new}} = (X_i - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}}) = (16 - 13) / (71 - 13) = .0517$

To normalize the third value of **19**, we would use the same formula:

- $X_{\text{new}} = (X_i - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}}) = (19 - 13) / (71 - 13) = .1034$

We can use this exact same formula to normalize each value in the original dataset to be between 0 and 1:

Data Values	Normalized
13	0
16	0.0517
19	0.1034
22	0.1552
23	0.1724
38	0.4310
47	0.5862
56	0.7414
58	0.7759
63	0.8621
65	0.8966
70	0.9828
71	1

Typically we **normalize** data when performing some type of analysis in which we have multiple variables that are measured on different scales and we want each of the variables to have the same range.

Standardization

It rescales a dataset to have a mean of 0 and a standard deviation of 1. It uses the following formula to do so:

$$x_{\text{new}} = (x_i - \bar{x}) / s$$

where:

- x_i : The i^{th} value in the dataset
- $\bar{x}$: The sample mean
- s : The sample standard deviation

Example: How to Standardize Data

Suppose we have the following dataset:

Data Values
13
16
19
22
23
38
47
56
58
63
65
70
71

The mean value in the dataset is 43.15 and the standard deviation is 22.13.

To normalize the first value of **13**, we would apply the formula shared earlier:

$$\bullet \mathbf{x_{new}} = (\mathbf{x_i} - \bar{\mathbf{x}}) / \mathbf{s} = (13 - 43.15) / 22.13 = \mathbf{-1.36}$$

To normalize the second value of **16**, we would use the same formula:

$$\bullet \mathbf{x_{new}} = (\mathbf{x_i} - \bar{\mathbf{x}}) / \mathbf{s} = (16 - 43.15) / 22.13 = \mathbf{-1.23}$$

To normalize the third value of **19**, we would use the same formula:

$$\bullet \mathbf{x_{new}} = (\mathbf{x_i} - \bar{\mathbf{x}}) / \mathbf{s} = (19 - 43.15) / 22.13 = \mathbf{-1.09}$$

We can use this exact same formula to standardize each value in the original dataset:

Data Values	Standardized
13	-1.36
16	-1.23
19	-1.09
22	-0.96
23	-0.91
38	-0.23
47	0.17
56	0.58
58	0.67
63	0.90
65	0.99
70	1.21
71	1.26

we typically **standardize** data when we'd like to know how many standard deviations each value in a dataset lies from the mean.

Data transformation with Binary coding

Binary Coding transforms categorical data to binary data

Categorical Data is data that corresponds to the Categorical Variable. A Categorical Variable is a variable that takes fixed, a limited set of possible values.

For example Gender, Blood group, a person having country residential or not, etc.

A **Binary Data** is a Data which uses two possible states or values i.e. 0 and 1.

Example 1.

Pandas is imported in order to use various inbuilt Functions available in Pandas framework

import pandas as pd

Data is initialized here

```
data = [{"Jagroop", "Male"}, {"Parveen", "Male"},
        {"Harjot", "Female"}, {"Pooja", "Female"},
        {"Mohit", "Male"}]
```

Data frame is created under column name Name and Gender

```
data_frame = pd.DataFrame(data, columns=["Name", "Gender"])
```

```
print(data_frame)
```

Data of Gender is converted into Binary Data

```
df_one = pd.get_dummies(data_frame["Gender"])
```

```
print(df_one)
```

Binary Data is Concatenated into Dataframe

```
df_two = pd.concat((df_one, data_frame), axis=1)
```

```
print(df_two)
```

Gendercolumn is dropped

```
df_two = df_two.drop(["Gender"], axis=1)
```

We want Male =0 and Female =1 So we drop Male column here

```
df_two = df_two.drop(["Male"], axis=1)
```

```
print(df_two)
```

Rename the Column

```
result = df_two.rename(columns={"Female": "Gender"})
```

Print the Result

```
print(result)
```

	Name	Gender
0	Jagroop	Male
1	Parveen	Male
2	Harjot	Female
3	Pooja	Female
4	Mohit	Male

	Female	Male
0	0	1
1	0	1
2	1	0
3	1	0
4	0	1

	Female	Male	Name	Gender
0	0	1	Jagroop	Male
1	0	1	Parveen	Male
2	1	0	Harjot	Female
3	1	0	Pooja	Female
4	0	1	Mohit	Male

	Female	Name
0	0	Jagroop
1	0	Parveen
2	1	Harjot
3	1	Pooja
4	0	Mohit

	Gender	Name
0	0	Jagroop
1	0	Parveen
2	1	Harjot
3	1	Pooja
4	0	Mohit

Example 2.

```
import pandas as pd
data=pd.read_csv('C:/Users/Hp/OneDrive/Desktop/Datasets/Mall_Customers.csv')
print(data.head())
```

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	1	Male	19	15	39
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
data=pd.get_dummies(data)
```

```
print(data.head())
```

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)	Gender_Female	\
0	1	19	15	39	0	
1	1	19	15	39	0	
2	3	20	16	6	1	
3	4	23	16	77	1	
4	5	31	17	40	1	

	Gender_Male
0	1
1	1
2	0
3	0
4	0

```
del data['Gender_Male']
```

```
print(data.head())
```

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)	Gender_Female
0	1	19	15	39	0
1	1	19	15	39	0
2	3	20	16	6	1
3	4	23	16	77	1
4	5	31	17	40	1

```
data.rename(columns={"Gender_Female": "Gender"})
```

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)	Gender
0	1	19	15	39	0
1	1	19	15	39	0
2	3	20	16	6	1
3	4	23	16	77	1
4	5	31	17	40	1
...	...	...	...	...	...
245	246	30	297	69	0
246	247	56	311	14	1
247	248	29	313	90	0
248	249	19	316	32	1
249	250	31	325	86	1

250 rows × 5 columns

Discretization:

It is a process of transforming continuous data into set of small intervals. Most Data Mining activities in the real world require continuous attributes. Yet many of the existing data mining frameworks are unable to handle these attributes.

For **example**, (1-10, 11-20) (age:- young, middle age, senior).

```
import pandas as pd
data=pd.read_csv('C:/Users/Hp/OneDrive/Desktop/Datasets/Student_Marks.csv')
print(data.head())
```

	StudentID	Age	Grade	Marks
0	1	19	1st	29
1	2	20	2nd	41
2	3	18	1st	57
3	4	21	2nd	29
4	5	19	1st	57

```
data['bucket']=pd.cut(data['Marks'],5,labels=['Poor','Below_average','Average','Above_Average','Excellent'])
data.head(10)
```

	StudentID	Age	Grade	Marks	bucket
0	1	19	1st	29	Poor
1	2	20	2nd	41	Poor
2	3	18	1st	57	Average
3	4	21	2nd	29	Poor
4	5	19	1st	57	Average
5	6	20	2nd	53	Below_average
6	7	19	3rd	78	Above_Average
7	8	21	3rd	70	Above_Average
8	9	22	3rd	97	Excellent
9	10	21	1st	58	Average